

СОФТУЕРНИ АРХИТЕКТУРИ

JAVASCRIPT OBJECT NOTATION



ВЪВЕДЕНИЕ В JSON

- JSON = JavaScript Object Notation
 - опростен формат за обмяна на данни;
 - удобен за работа както за хората, така и за компютрите;
 - базиран на едно подмножество на езика за програмиране JavaScript;
 - има текстов формат, напълно независим от реализацията на езика;
 - използва конвенции, познати на програмистите на C-подобните езици: C, C++, C#, Java, JavaScript, Perl, Python, и др.;

СТРУКТУРА НА JSON

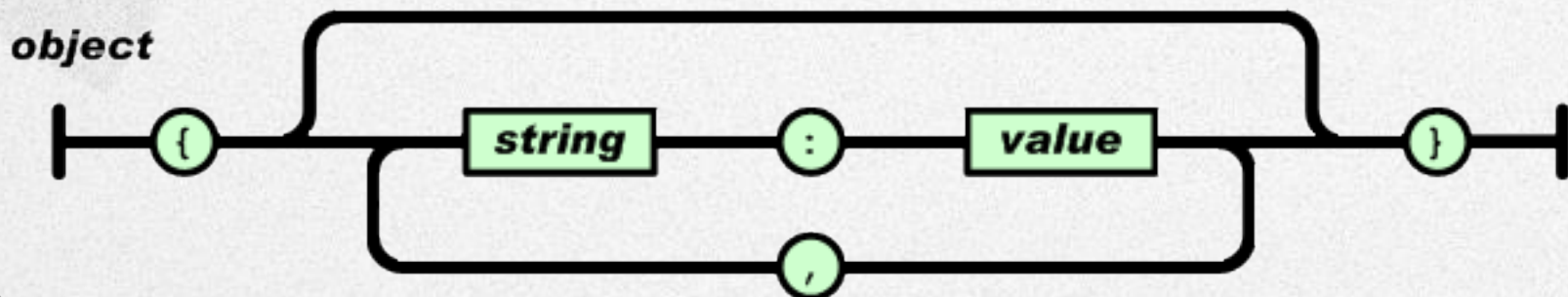
- JSON се състои от две структури:
 - колекция от двойки име/стойност;
 - подреден списък от стойности.
- Универсални структури от данни.
- Поддържат се от всички модерни езици под една или друга форма.
- Подходящ за обмяна на данни между езиците за програмиране.

СТРУКТУРА: КОЛЕКЦИЯ ОТ ДВОЙКИ ИМЕ/СТОЙНОСТ

- В различните езици, това се реализира като:
 - **обект**
 - запис
 - структура
 - речник
 - хеш таблица
 - именуван списък
 - асоциативен масив.

ОБЕКТ: КОЛЕКЦИЯ ОТ ДВОЙКИ ИМЕ/СТОЙНОСТ

- Един обект е не подреден набор от двойки име/стойност.
- Обекта започва с { (лява фигурна скоба) и завършва с } (дясна фигурна скоба).
- След всяко име се поставя : (двуеточие), а двойките име/стойност се разделят със , (запетайка).



ОБЕКТ: КОЛЕКЦИЯ ОТ ДВОЙКИ ИМЕ/СТОЙНОСТ

```
{"firstName": "Иван", "lastName": "Петров"}
```

```
{  
    "firstName" : "Иван",  
    "lastName"  : "Петров"  
}
```

```
<person>  
  <firstName>Иван</firstName>  
  <lastName>Петров</lastName>  
</person>
```

ЙЕРАРХИЯ ОТ ОБЕКТИ

```
{  
  "name": {  
    "firstName" : "Иван",  
    "lastName"  : "Петров"  
  },  
  "address": {  
    "city"      : "Пловдив",  
    "street"    : "Хр. Ботев"  
  }  
}
```

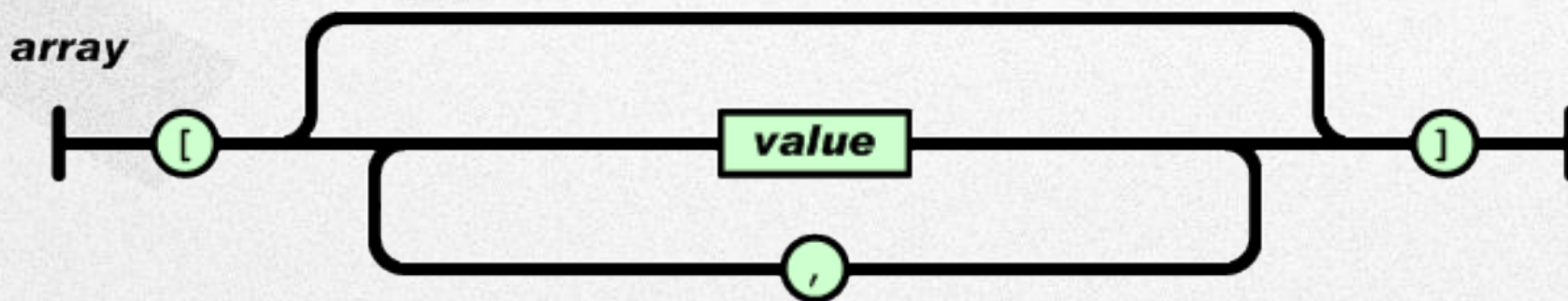
```
<person>  
  <name>  
    <firstName>Иван</firstName>  
    <lastName>Петров</lastName>  
  </name>  
  <address>  
    <city>Пловдив</city>  
    <street>Хр. Ботев</street>  
  </address>  
</person>
```

СТРУКТУРА: ПОДРЕДЕН СПИСЪК ОТ СТОЙНОСТИ

- В повечето езици, това се реализира чрез:
 - **масив**
 - вектор
 - списък
 - последователност

МАСИВ

- Един масив е подредена колекция от стойности.
- Масивът започва с [(лява квадратна скоба) и завършва с] (дясна квадратна скоба).
- Стойностите се разделят чрез , (запетайка).



МАСИВ

```
["червен", "зелен", "син", "бял", "черен"]
```

```
<colors>  
  <color>червен</color>  
  <color>зелен</color>  
  <color>син</color>  
  <color>бял</color>  
  <color>черен</color>  
</colors>
```

МАСИВ

```
{
  "name"    : {
    "firstName" : "Иван",
    "lastName"  : "Петров"
  },
  "address": {
    "city"      : "Пловдив",
    "street"    : "Хр. Ботев"
  },
  "fn"       : ["1234567890", "0987654321"]
}
```

МАСИВ ОТ ОБЕКТИ

```
[  
  {"firstName": "Иван",      "lastName": "Петров",      "age": 20},  
  {"firstName": "Николай",   "lastName": "Иванов" ,      "age": 25},  
  {"firstName": "Мария",     "lastName": "Николова",     "age": 22}  
]
```

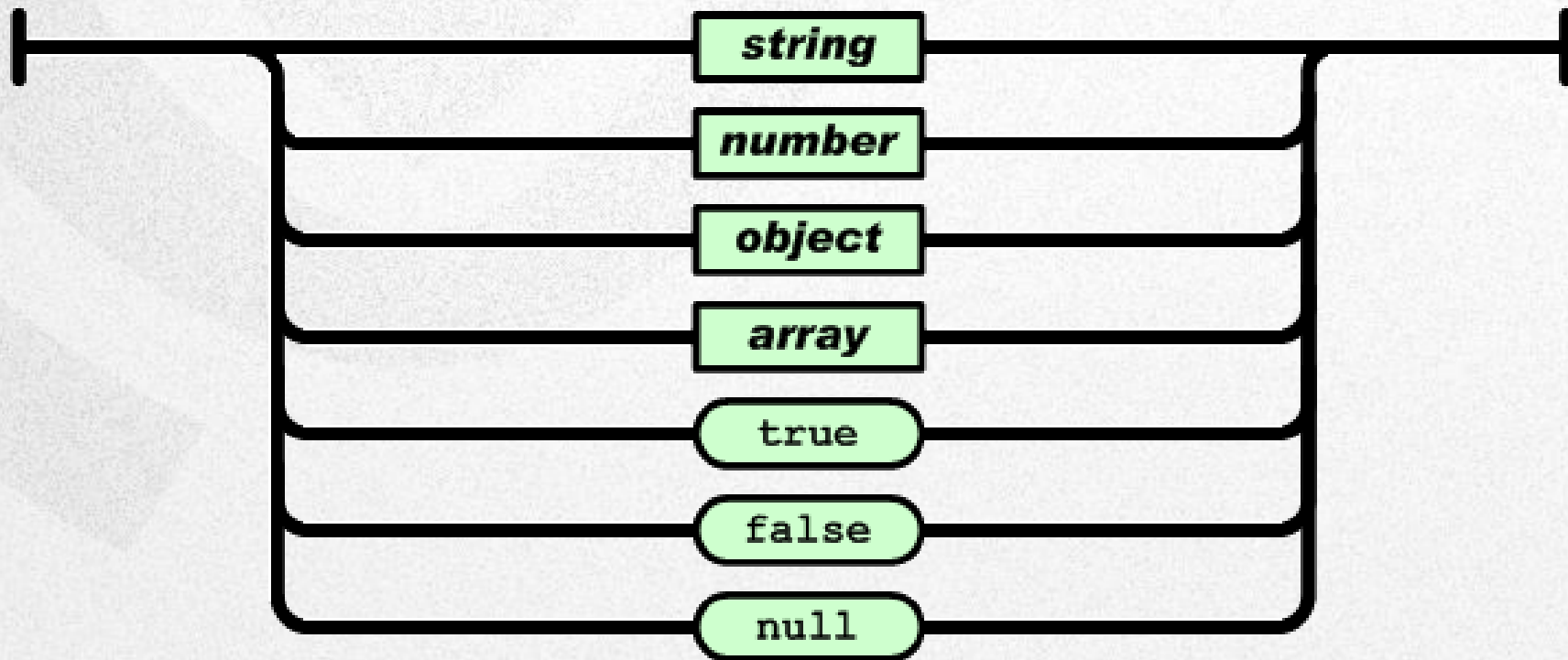
```
{ "employees":  
  [  
    {"firstName": "Иван",      "lastName": "Петров",      "age": 20},  
    {"firstName": "Николай",   "lastName": "Иванов" ,      "age": 25},  
    {"firstName": "Мария",     "lastName": "Николова",     "age": 22}  
  ]  
}
```

ВЪЗМОЖНИ СТОЙНОСТИ

- Една стойност може да бъде:
 - низ в двойни кавички
 - число
 - истина (true) или лъжа (false)
 - null
 - обект
 - масив
- Тези структури могат да бъдат вложени.

ВЪЗМОЖНИ СТОЙНОСТИ

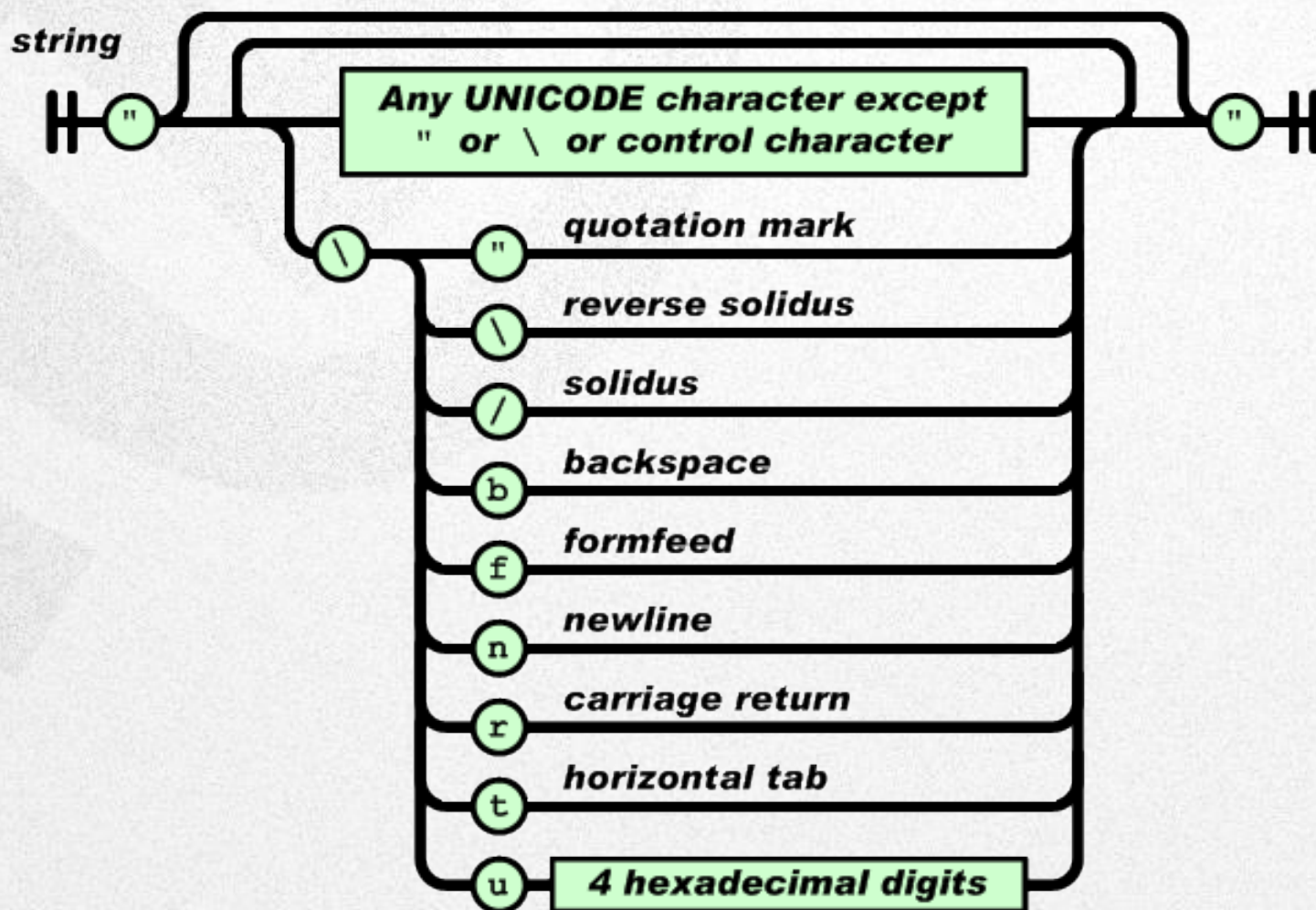
value



НИЗ

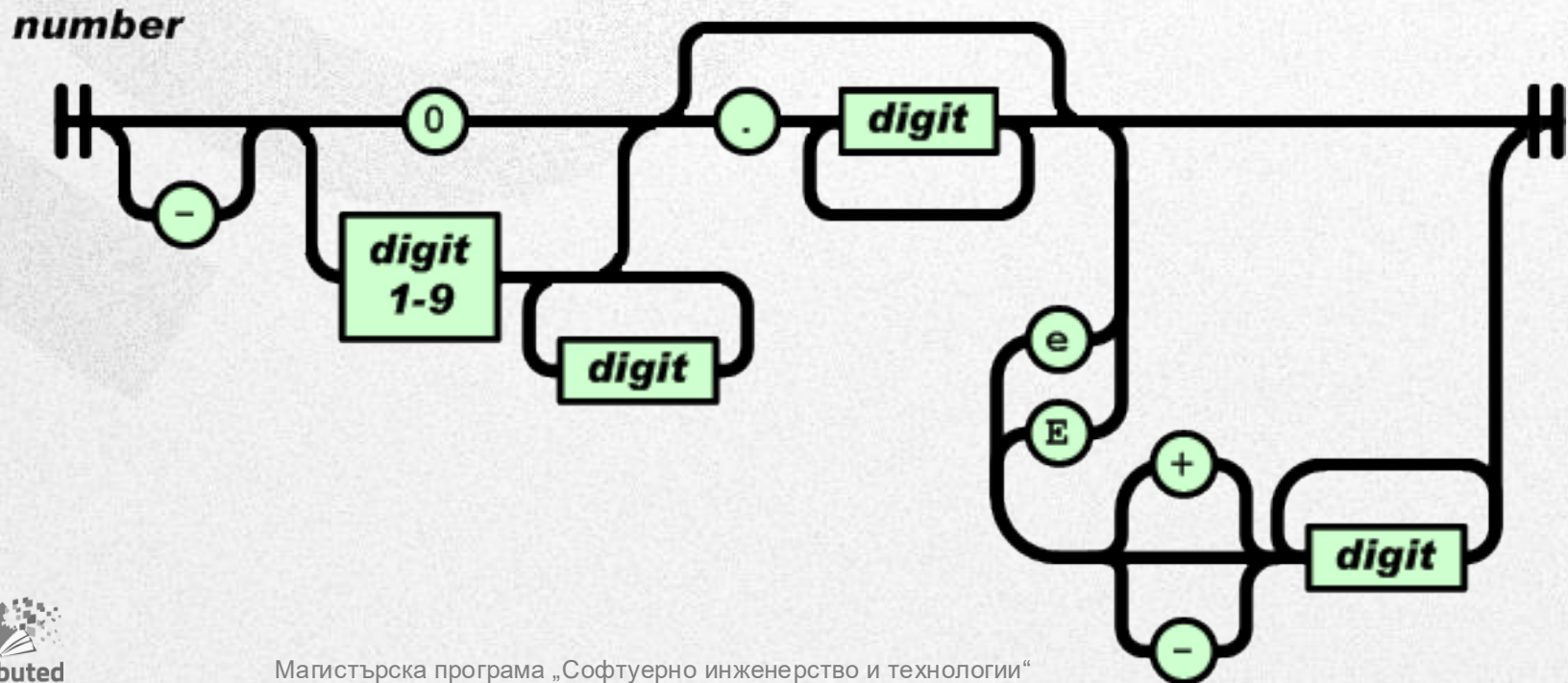
- **НИЗ** е колекция от от нула или повече Unicode СИМВОЛИ
- обграден от двойни кавички
- \ (обратна наклонена черта) - escape символ
- **СИМВОЛ** се представя чрез низ, съдържащ само един СИМВОЛ.
- низът е подобен на тези в C или Java.

НИЗ



ЧИСЛО

- Числото много наподобява числото в C или Java, с изключение на това, че осмичният и шестнадесетичен формат не се използват.



ПРАЗНО ПРОСТРАНСТВО

- Между двойките може да бъде вмъквано празно пространство (whitespace):
 - <TAB>
 - <CR>
 - <LF>
 - <SP>

ИЗПОЛЗВАНЕ В JAVASCRIPT

```
var employees = [  
  {"firstName": "Иван",      "lastName": "Петров",      "age": 20},  
  {"firstName": "Николай",  "lastName": "Иванов" ,      "age": 25},  
  {"firstName": "Мария",    "lastName": "Николова",    "age": 22}  
];  
  
employees[0].firstName + " " employees[0].lastName; // Иван Петров  
  
employees[0].firstName = "Ванката";
```

JAVASCRIPT JSON OBJECT

- Обектът JSON е единичен обект, който има две функции:
 - **parse** – за парсиране на JSON текст;
 - **stringify** – за конструиране на JSON текст.
- Разлики спрямо стандартен обект в JavaScript:
 - Не е възможно JSON обекта да се използва като конструктор с оператора *new*.
 - Не е възможно JSON обекта да се извиква като функция.

PARSE (TEXT [, REVIEWER])

- `parse (text [, reviewer])`
 - Парсира JSON текст и създава JavaScript стойност, където:
 - JSON обектите се реализират като JavaScript обекти;
 - JSON масивите се реализират като JavaScript масиви;
 - JSON низове, числа, логически тип и null се реализират като JavaScript низове, числа, boolean и null.

PARSE (TEXT [, REVIEWER])

- Опционалният *reviver* параметър е функция, която приема два параметъра – *key* и *value*.
 - Чрез нея може да се филтрира и променя резултата.
 - Функцията се извиква за всяка двойка *key/value*, намираща се в текста и стойността, която върне се използва вместо оригиналната стойност.
 - Ако функцията връща това което получава – крайният резултат остава не променен.
 - Ако функцията върне **undefined** – атрибутът се премахва от резултата.

ИЗПОЛЗВАНЕ В JAVASCRIPT

```
var txt = '{"employees": [' +  
    '{"firstName": "Иван",      "lastName": "Петров",      "age": 20}, ' +  
    '{"firstName": "Николай",  "lastName": "Иванов" ,      "age": 25}, ' +  
    '{"firstName": "Мария",    "lastName": "Николова",      "age": 22} ' +  
    ']]';  
  
var obj = JSON.parse(txt);  
  
// Резултатът е "Иван Петров"  
obj.employees[0].firstName + " " obj.employees[0].lastName;
```

STRINGIFY (VALUE [, REPLACER [, SPACE]])

- `stringify (value [, replacer [, space ]])`
 - Връща низ в JSON формат, който представя JavaScript стойност (параметъра `value`).
 - Първият параметър – **value**, е задължителен. Може да приема:
 - обект
 - масив
 - допуска се и: низ, `boolean`, число, `null`
 - Параметърът **replacer** е опционален. Може да съдържа:
 - функция, която променя начина, по който обектите и масивите се преобразуват в JSON низ;
 - масив от низове и числа, които служат като списък на позволените атрибути (*white list*), които ще бъдат включени в JSON низа

STRINGIFY (VALUE [, REPLACER [, SPACE]])

- Параметърът **space** е опционален и може да бъде низ или число.
 - указва в резултата да бъдат вмъкнати празни пространства (white spaces) за повишаване на читаемостта от човек.

JSON VS. XML

- Прилики:
 - представя се в чист текст
 - самоописателен (разбираем и от човек)
 - йерархичен
 - може да бъде парсиран от JavaScript
 - може да бъде транспортиран през HTTP (AJAX)

JSON VS. XML

- Разлики:
 - няма затварящ таг
 - по-малко обем на допълнителната информация
 - по-бърз за четене и създаване
 - може да бъде парсиран от вграждения в JavaScript `eval()`
 - ИЗПОЛЗВА МАСИВИ
 - няма запазени думи

JSON VS. XML

- Обработка:
 - XML:
 1. извличане на XML от XML документ;
 2. използване на XML DOM за обхождане на документа;
 3. извличане на стойностите и записването им в променливи/обекти.
 - JSON:
 1. извличане на JSON низ;
 2. парсиране с `eval()`